

Arboles

Los árboles son estructuras de datos jerárquicas que consisten en nodos conectados por aristas. Se utilizan ampliamente en ciencias de la computación para diversas aplicaciones, como organizar datos, permitir búsquedas más rápidas y representar relaciones jerárquicas. Aquí tienes un resumen centrado en los principales tipos de árboles:

1. Árbol binario

- Un árbol en el que cada nodo tiene como máximo dos hijos, conocidos como hijo izquierdo y hijo derecho.
- Propiedades:
 - La altura puede variar.
 - No hay un orden específico para los nodos.

2. Árbol de búsqueda binaria (BST)

- Un árbol binario con la propiedad adicional de que para cualquier nodo dado:
 - El subárbol izquierdo contiene nodos con valores menores que el valor del nodo.
 - El subárbol derecho contiene nodos con valores mayores que el valor del nodo.
- Propiedades:
 - Permite operaciones de búsqueda, inserción y eliminación eficientes ($O(\log n)$ en promedio).

3. Árboles balanceados

- Un tipo de árbol binario que mantiene una estructura balanceada para garantizar operaciones eficientes.
- Tipos:
 - Árbol AVL: Un BST autoequilibrado donde la diferencia de alturas entre los subárboles izquierdo y derecho es como máximo uno.
 - Árbol Rojo-Negro: Un BST autoequilibrado con propiedades que garantizan que la altura del árbol sea logarítmica en relación con el número de nodos.

4. Árbol binario completo

- Un árbol binario en el que todos los niveles, excepto posiblemente el último, están completamente llenos, y todos los nodos están tan a la izquierda como sea posible.
- Propiedades:
 - Uso eficiente del espacio y a menudo se utiliza en implementaciones de montículos.

5. Árbol binario completo

- Un árbol binario en el que cada nodo tiene 0 o 2 hijos.
- Propiedades:
 - El número total de nodos siempre es $2^h - 1$ para un árbol de altura h .

6. Árbol n-ario

- Un árbol en el que cada nodo puede tener como máximo N hijos.
- Propiedades:
 - Más general que los árboles binarios y se utiliza para representar jerarquías más complejas.

7. Trie (Árbol de prefijos)

- Un tipo especial de árbol utilizado para almacenar un conjunto dinámico de cadenas, donde las claves suelen ser cadenas.
- Propiedades:
 - Cada nodo representa un carácter y los caminos en el árbol representan diferentes cadenas; eficiente para búsquedas de prefijos.

8. Árbol de segmentos

- Un árbol utilizado para almacenar intervalos o segmentos; útil para responder consultas de rango de manera eficiente.
- Propiedades:
 - Permite consultas rápidas sobre información de matrices, como sumas o mínimos en un rango.

9. Árbol de Fenwick (Árbol binario indexado)

- Una estructura de datos similar a un árbol que proporciona métodos eficientes para tablas de frecuencia acumulativa.
- Propiedades:
 - Permite actualizaciones y consultas de sumas de prefijos en tiempo logarítmico.

10. Árbol B

- Una estructura de datos de árbol autoequilibrado que mantiene datos ordenados y permite búsquedas, acceso secuencial, inserciones y eliminaciones en tiempo logarítmico.
- Propiedades:
 - Se utiliza en bases de datos y sistemas de archivos debido a su capacidad para manejar eficientemente el almacenamiento basado en disco.

Los árboles son estructuras versátiles que varían en complejidad y aplicación. Comprender sus tipos y propiedades ayuda a seleccionar la estructura adecuada para tareas computacionales específicas, desde una simple organización de datos hasta una compleja indexación de bases de datos.

Ejercicios

1. Recorridos de árboles binarios: Implementa métodos de recorrido en orden, preorden y postorden para un árbol binario.
2. Cálculo de la altura del árbol: Escribe una función para calcular la altura de un árbol binario.
3. Recorrido por niveles: Implementa una función para recorrer un árbol binario por niveles (BFS).
4. Árbol balanceado: Escribe una función para determinar si un árbol binario está balanceado en altura.
5. Profundidad máxima: Encuentra la profundidad máxima de un árbol binario utilizando recursión.

6. Ancestro común más bajo: Implementa una función para encontrar el ancestro común más bajo de dos nodos en un árbol de búsqueda binaria (BST).
 7. Árbol espejo: Crea una función que convierta un árbol binario en su imagen espejo.
 8. Validación de árbol de búsqueda binaria: Escribe una función para verificar si un árbol binario es un árbol de búsqueda binaria válido.
 9. Encontrar el k-ésimo elemento más pequeño: Implementa un método para encontrar el k-ésimo elemento más pequeño en un BST.
 10. Serialización/Deserialización de árbol: Escribe funciones para serializar y deserializar un árbol binario.
 11. Invertir un árbol binario: Implementa una función para invertir un árbol binario.
 12. Suma de nodos hoja: Escribe una función para calcular la suma de todos los nodos hoja en un árbol binario.
 13. Diámetro de un árbol: Implementa una función para encontrar el diámetro de un árbol binario dado.
 14. Recorrido en orden vertical: Escribe una función para realizar un recorrido en orden vertical de un árbol binario.
 15. Construir un árbol binario a partir de recorridos en preorden/inorden: Implementa una función para reconstruir un árbol binario a partir de sus recorridos en preorden e inorden.
-

Grafos

Los grafos son estructuras de datos fundamentales utilizadas para modelar relaciones y conexiones entre pares de objetos. Consisten en nodos (vértices) y aristas que conectan pares de estos nodos. Los grafos se utilizan ampliamente en diversos dominios, como redes, transporte, redes sociales y muchas otras aplicaciones.

Componentes clave de los grafos

1. Vértices (Nodos): Los puntos o entidades individuales en un grafo. Cada vértice puede representar un objeto, como una ciudad, una persona o cualquier elemento discreto.
2. Aristas (Enlaces): Conexiones entre pares de vértices. Pueden denotar relaciones, caminos o flujos entre los nodos.

Tipos de grafos

1. Grafo dirigido (Digrafo):
 - En un grafo dirigido, las aristas tienen una dirección, lo que indica una relación unidireccional de un vértice a otro.
 - Ejemplo: Un grafo de seguidores de Twitter, donde la persona A puede seguir a la persona B, pero no viceversa.
2. Grafo no dirigido:
 - En un grafo no dirigido, las aristas no tienen dirección; la relación es mutua.

- Ejemplo: Un grafo de amistad, donde si la persona A es amiga de la persona B, entonces B también es amiga de A.

3. Grafo ponderado:

- En un grafo ponderado, cada arista tiene un peso o costo asociado. Esto se utiliza comúnmente para representar distancias o costos para recorrer de un vértice a otro.
- Ejemplo: Una red de carreteras donde las aristas representan distancias entre ciudades.

4. Grafo no ponderado:

- En un grafo no ponderado, las aristas no tienen pesos; todas las conexiones se consideran iguales.

5. Grafo simple:

- Un grafo sin aristas múltiples entre vértices y sin bucles (aristas que conectan un vértice consigo mismo).

6. Multigrafo:

- Un grafo que permite múltiples aristas entre el mismo conjunto de vértices y/o bucles.

7. Grafo completo:

- Un grafo en el que cada par de vértices está conectado por una arista única.
- Notación: Un grafo completo con n vértices se denota como K_n .

8. Grafo cíclico:

- Un grafo que contiene al menos un ciclo, lo que significa que hay un camino que comienza y termina en el mismo vértice.

9. Grafo acíclico:

- Un grafo que no contiene ciclos. Un caso especial de un grafo acíclico es un Grafo Acíclico Dirigido (DAG), que se utiliza a menudo en programación y representación de dependencias.

10. Grafo conectado:

- Un grafo no dirigido es conectado si hay un camino entre cada par de vértices.

11. Grafo desconectado:

- Un grafo en el que al menos dos vértices no tienen un camino que los conecte.

Representaciones de grafos

1. Matriz de adyacencia:

- Una matriz 2D donde el elemento en la fila i y la columna j indica si hay una arista desde el vértice i al vértice j .
- Complejidad espacial: $O(V^2)$, donde V es el número de vértices.

2. Lista de adyacencia:

- Un arreglo de listas donde cada índice representa un vértice y contiene una lista de vértices adyacentes.
- Complejidad espacial: $O(V + E)$, donde E es el número de aristas.

3. Lista de aristas:

- Una lista de todas las aristas del grafo, cada una representada como un par (o tupla) de vértices.
- Útil para grafos dispersos.

Aplicaciones de los grafos

- Redes sociales: Modelar relaciones, interacciones y conexiones entre usuarios.
- Algoritmos de enrutamiento: Encontrar el camino más corto en enrutamiento de redes (por ejemplo, el algoritmo de Dijkstra, búsqueda A^*).
- Resolución de dependencias: Gestionar tareas basadas en dependencias previas (por ejemplo, programación de proyectos).
- Sistemas de recomendación: Utilizar relaciones entre usuarios y elementos para sugerir nuevos elementos o conexiones.
- Análisis de redes: Estudiar la estructura y el comportamiento de redes complejas, como Internet o sistemas biológicos.

Los grafos son estructuras de datos poderosas que representan eficazmente relaciones y conexiones entre entidades. Comprender sus tipos, representaciones y aplicaciones es crucial para resolver muchos problemas computacionales en diversos campos como ciencias de la computación, investigación de operaciones y ciencias sociales.